Instituto Superior de Engenharia de Lisboa (ISEL)
Departamento de Engenharia Eletrónica e de Telecomunicações e Computadores (DEETC)

LEIM
Licenciatura em Engenharia Informática e Multimédia
Unidade Curricular de Projeto

Sistema de Resumos de Processos Crime



Gonçalo Barbosa (49845)

[redacted]

Orientadores

Professor [redacted]

Inspetor Chefe [redacted]

Junho, 2025

Resumo

Resumir processos crime é uma tarefa lenta e por vezes repetitiva. Os métodos atuais envolvem a exigência de leitura, interpretação jurídica, tempo e mão de obra especializada.

O sistema proposto utiliza *Large Language Models* para gerar relatórios finais de forma automática e rápida quando comparada aos métodos anteriores.

Além de acelerar significativamente o processo, o sistema garante maior consistência nas sínteses produzidas e serve como ferramenta de apoio à decisão jurídica.

Através de exemplos e alimentado por documentos de processos crime reais, o sistema cria sínteses claras e objetivas focadas nos relatórios criminais.

Palavras-chave: processos crime, inteligência artificial, relatórios jurídicos, sumarização de documentos, automatização.

Abstract

Summarizing criminal cases is a slow and often repetitive task. Current methods require legal reading, interpretation, time, and specialized human resources.

The proposed system uses *Large Language Models* to automatically and rapidly generate final reports, offering a significant improvement over traditional methods.

In addition to significantly speeding up the process, the system ensures greater consistency in the summaries produced and serves as a tool to support legal decision-making.

Trained on examples and real criminal case documents, the system produces clear and objective summaries focused on criminal reports.

Keywords: criminal cases, artificial intelligence, legal reports, document summarization, automation.

Agradecimentos

Gostaríamos, inicialmente, de agradecer ao [redacted], pela proposta cativante e por toda a mentoria e auxílio durante o desenvolvimento do projeto.

De seguida, também queríamos expressar a nossa gratidão ao Professor Doutor [redacted] pela arguência do nosso projeto e pelas recomendações que nos foram concedidas.

Agradecemos profundamente ao Inspetor [redacted] e à organização da Polícia Judiciária, [redacted] por nos terem disponibilizado um espaço para o nosso crescimento autónomo e para o progresso da proposta de projeto.

Finalmente, gostaríamos imensamente de agradecer a ambas as nossas famílias por nos terem dado todo o apoio incondicional durante o desenvolvimento desta nova etapa.

[redacted]

Índice

Resumo **i**

Abstract **iii**

Agradecimentos **v**

Índice **ix**

Lista de Tabelas **xi**

Lista de Figuras **xiii**

1 Introdução **1**

2 Trabalho Relacionado **3**

2.1 Fundamentação Teórica 3

2.1.1 Processos Crime 3

2.1.2 Inteligência Artificial 3

2.1.3 *Large Language Models* 4

2.1.4 *Ollama* 4

2.1.5 *Flask* 5

2.2 *Prompt Engineering* 5

2.2.1 *Zero-Shot Learning* 5

2.2.2 *One-Shot Learning* 6

2.2.3 *Few-Shot Learning* 6

2.3 *Docker* 6

2.4 Trabalho Relacionado 7

3 Modelo Proposto **9**
3.1 Requisitos 9
3.1.1 Requisitos Funcionais 9
3.1.2 Requisitos Não Funcionais 10
3.2 Casos de Utilização 11
3.3 Abordagem 14

4 Implementação do Modelo **17**
4.1 Implementação da Solução Desenvolvida 17
4.1.1 Interface Web e Experiência do Utilizador 18
4.1.2 Arquitetura *Backend* 22

5 Validação e Testes **29**
5.1 Geração de Relatórios Finais 29

6 Conclusões e Trabalho Futuro **37**
6.1 Conclusões 37
6.2 Trabalho Futuro 38

A Arquitetura **41**

B Agente Capa **43**

C Agente Cota **47**

D Agente Inquirição **51**

E Agente Inquirição **55**

F Agente Notificação ao Lesado **57**

G Agente Informação **59**

H Agente Resumo Cronológico **61**

Bibliografia **65**

Lista de Tabelas

3.1 Tabela de Requisitos funcionais 10
3.2 Tabela de Requisitos funcionais 10

Lista de Figuras

3.1 Modelo de Casos de Utilização 12

4.1 Página Inicial da Aplicação *Web* 18
4.2 Ficheiros Submetidos 19
4.3 Visualização do Ficheiro submetido 19
4.4 Respostas Geradas pelo agente criado 20
4.5 Possível visualização e edição das respostas geradas 21
4.6 Possível visualização, edição e *download* do Relatório Final . . 22
4.7 Frontend desenvolvido 24
4.8 Backend desenvolvido 25

A.1 UML 42

Chapter 1

Introdução

O sistema proposto, denominado de Sistema de Resumos de Processos Crime, é um projeto cujo objetivo é sintetizar processos crime. O acesso digital aos processos crime por parte dos inspetores da Polícia Judiciária constitui uma necessidade crescente no contexto da modernização e otimização das atividades de investigação criminal. Este projeto propõe o desenvolvimento de um sistema que, recorrendo à execução de agentes baseados em *Large Language Models*, doravante designados por *LLMs*, permite apoiar de forma eficiente a análise de documentos e a produção automática de relatórios criminais finais, assegurando rigor técnico, coesão e ganhos significativos em termos de produtividade.

O acréscimo da quantidade de processos crime representa um dos principais desafios operacionais enfrentados pela Polícia Judiciária, sendo esta uma principal causa que exige uma mudança no que toca à gestão de processamento de dados jurídicos. Neste contexto, identificou-se uma necessidade de otimizar a produtividade no trabalho dos inspetores, especificamente nas fases de leitura, interpretação e produção de relatórios criminais.

O projeto envolveu a implementação de uma plataforma web que permite a inserção e gestão digital de processos crime. Com recurso a agentes *AI* suportados em *LLMs*, foram exploradas diversas configurações de *system prompts*, cuja formulação e otimização foram ajustadas para obter resultados linguisticamente consistentes, tecnicamente relevantes e adequados ao contexto formal exigido. Para isso, foram também utilizados múltiplos processos já existentes fornecidos para recolher dados relevantes e avaliar o desempenho do sistema.

Este relatório vai consistir numa estrutura organizada por capítulos, de forma a guiar o leitor, de modo progressivo e fundamentado, através das várias etapas que compõem o desenvolvimento deste projeto. O **Capítulo 1** é onde o leitor se encontra no momento, e visa a descrição e a introdução dos conteúdos do desenvolvimento do projeto. No **Capítulo 2**, são apresentados os principais fundamentos teóricos e tecnológicos que sustentam toda a evolução do projeto, passando por diversos conceitos, tal como processos crime, *prompt engineering*, *LLMs* e *Generative AI*, pela descrição das linguagens e bibliotecas utilizadas e pelo detalhe da plataforma *Docker*. O **Capítulo 3** incide sobre o modelo proposto de desenvolvimento, detalhando a estrutura funcional da aplicação, os seus casos de utilização e respetivos requisitos. No **Capítulo 4**, é aprofundada uma explicação sobre a implementação do projeto, que passa por todas as componentes desenvolvidas (*frontend* e *backend*), a integração dos agentes *LLM* e os procedimentos de otimização aplicados. Inclui-se ainda uma descrição das abordagens utilizadas na definição dos *prompts* para os agentes. De seguida, o **Capítulo 5** apresenta uma fase de testes, validações e análise crítica de todos os resultados obtidos, conseguindo evidenciar o desempenho positivo dos agentes criados, no que toca ao objetivo do projeto. Finalmente, o **Capítulo 6** vai sintetizar as conclusões do projeto desenvolvido e apresentar um conjunto de perspetivas de desenvolvimento futuro, especialmente em termos de escalabilidade e possíveis melhorias do sistema.

Ao longo do documento, adotaram-se as seguintes convenções: itálico para estrangeirismos e "aspas" para citações.

Chapter 2

Trabalho Relacionado

Um dos principais objetivos no desenvolvimento deste projeto é explorar o potencial da Inteligência Artificial, mais especificamente dos modelos de linguagem de última geração, neste caso *LLMs*, na análise de processos crime. A ideia principal é conseguir utilizar estes agentes para auxílio na leitura, interpretação e organização da grande quantidade de informação contida em inquéritos e processos investigativos.

Para melhor compreensão do projeto desenvolvido, foram introduzidos alguns fundamentos teóricos que foram considerados fundamentais, tal como a menção de dois exemplos de trabalhos relacionados.

2.1 Fundamentação Teórica

2.1.1 Processos Crime

Um processo crime é o conjunto de atos processuais destinados à investigação, instrução e julgamento de factos que possam constituir um crime. Trata-se de um procedimento formal conduzido pelas autoridades judiciárias e policiais, com o objetivo de apurar a veracidade dos acontecimentos, identificar os autores e garantir que a justiça penal seja aplicada de forma equitativa [da República, site].

2.1.2 Inteligência Artificial

O conceito de Inteligência Artificial [Russell e Norvig, 2020] refere-se à capacidade de sistemas computacionais executarem diversas tarefas que nor-

malmente exigem inteligência humana. O objetivo desta nova tecnologia visa o desenvolvimento de máquinas e algoritmos que sejam capazes de aprender com dados reais, tomar decisões, fazer previsões e resolver problemas de forma autónoma. Os sistemas de inteligência artificial visam simular processos cognitivos humanos, como a análise de uma enorme quantidade de informação, identificação de padrões, desempenhando funções complexas de uma forma melhorada e mais rápida.

2.1.3 *Large Language Models*

Os *Large Language Models*, *LLMs*, são uma forma avançada de Inteligência Artificial que se caracteriza pelo uso de enormes quantidades de dados textuais, provenientes de livros, *websites* e qualquer outro tipo de repositórios públicos, para treino dos modelos. Estes modelos usufruem da aquisição da compreensão, geração e manipulação de uma linguagem natural semelhante à de um ser humano [Naveed et al., 2024].

2.1.4 *Ollama*

Para garantir a execução eficiente e local dos *LLMs*, recorreu-se à ferramenta *Ollama* [Ollama, 2023]. Esta plataforma permite carregar e utilizar modelos de linguagem diretamente num ambiente local, eliminando a necessidade de dependência de serviços externos e a exposição online de processos crime. O *Ollama* permite um maior controlo sobre os dados processados e uma maior segurança na manipulação de conteúdos.

O *Ollama* suporta a utilização de múltiplos *LLM*. O modelo utilizado foi o *gemma* 3, sendo que o mesmo foi escolhido tendo em conta as capacidades limitadas das máquinas utilizadas. Este modelo oferece quatro versões, mil milhões (1B), quatro mil milhões (4B), doze mil milhões (12B) ou vinte e sete mil milhões (27B) de parâmetros *multimodal*, sendo que a segunda referida foi a integrada no projeto [Ollama, 2024]. A escolha deste modelo deve-se ao seu desempenho superior quando comparado com outras alternativas na mesma faixa de capacidade e ao seu elevado contexto face a outras *LLMs* ($128k$).

2.1.5 *Flask*

A biblioteca *Flask* [Flask, 2025] "foi concebida para facilitar o início do desenvolvimento de forma rápida e simples, com capacidade para escalar até aplicações complexas" [1] ([Flask2.0.1, 2021], tradução própria). A biblioteca permite gerir rotas e lidar com pedidos e respostas, funcionando como base de comunicação entre o cliente, *frontend*, e o servidor, *backend*. Através do seu sistema de templates, foi possível gerir páginas *HTML* dinâmicas.

2.2 *Prompt Engineering*

No contexto deste projeto, recorreu-se a técnicas de *prompt engineering*, sendo um processo que consiste na elaboração estruturada de instruções fornecidas aos *LLMs*, com o objetivo de maximizar a precisão e contextualização das respostas que são geradas.

A construção dos *prompts* permite, assim, controlar melhor a qualidade das respostas, reduzir ambiguidades e assegurar que a informação gerada responde de forma clara e objetiva às necessidades específicas do sistema.

Existem diversos tipos de técnicas de *prompt engineering*. No desenvolvimento deste projeto, foram abordadas as técnicas *Zero-Shot Learning*, *One-Shot Learning* e *Few-Shot Learning* [Sekiri, 2024, Aryani, 2023].

2.2.1 *Zero-Shot Learning*

Zero-Shot Learning refere-se à capacidade dos *LLMs* realizarem diversas tarefas novas sem que se forneçam certos tipos de exemplos ou treinos específicos para o resultado dessas tarefas. Esta técnica assenta apenas no conhecimento do modelo, sendo suficiente fornecer unicamente instruções objetivas.

Para que a técnica de *Zero-Shot Learning* funcione da forma correta, alguns fatores têm de ser considerados. Primeiramente, é fundamental que o *prompt* fornecido ao modelo seja claro, objetivo e suficientemente descritivo da tarefa pretendida, dependendo do contexto em que está a ser inserido. O facto de que nenhum exemplo concreto seja fornecido implica que o modelo deve compreender a tarefa apenas com base na descrição textual da instrução.

[1] "It is designed to make getting started quick and easy, with the ability to scale up to complex applications"

A escolha de vocabulário e estrutura na criação do *prompt* irá influenciar de forma significativa a qualidade da resposta. Por fim, a avaliação dos resultados deve ser feita com critérios bem definidos, dado que a ausência de exemplos pode introduzir maior variabilidade nas respostas geradas.

Adicionalmente, é importante considerar que, apesar da sua flexibilidade, a técnica *Zero-Shot Learning* pode produzir respostas menos precisas ou contextualmente inadequadas em tarefas complexas. Nestes casos, torna-se recomendável redefinir o *prompt*, ou mesmo recorrer a técnicas complementares como o *Few-Shot Learning*, quando a tarefa exigir maior controlo sobre o formato ou o conteúdo da resposta.

2.2.2 *One-Shot Learning*

One-Shot Learning corresponde a uma abordagem em que o modelo recebe um único exemplo de entrada e saída antes de ser solicitado a realizar uma tarefa. Este exemplo serve como referência para o modelo compreender o formato esperado e o contexto da tarefa, permitindo generalizar a partir desse único exemplo.

2.2.3 *Few-Shot Learning*

A ténica *Few-Shot Learning* consiste em fornecer ao modelo de linguagem alguns exemplos, normalmente entre dois a cinco, para averiguar a tarefa que se pretende executar. Estes exemplos funcionam como guias, para permitir que o modelo consiga identificar padrões, estilo e estrutura das respostas esperadas.

2.3 *Docker*

O *Docker* "fornece uma interface gráfica simples e intuitiva que permite gerir os seus contentores, aplicações e imagens diretamente a partir da própria máquina" [Docker, 2013].

É uma ferramenta que facilita o desenvolvimento, execução e gestão de aplicações através de um sistema de contentores. Estes contentores permitem isolar a aplicação, com todas as suas dependências, do sistema operativo subjacente, criando um ambiente de execução autónomo e replicável. Esta

abordagem assegura que a aplicação se comporte de forma consistente em diferentes ambientes, eliminando problemas de incompatibilidade entre sistemas de desenvolvimento, teste e produção.

2.4 Trabalho Relacionado

Apesar do desenvolvimento do projeto ser focado na área da criminologia, existem diversos outros projetos de investigação que recorrem a metodologias semelhantes, mas que partilham o mesmo objetivo fundamental: a automatização da análise e interpretação de documentos complexos através de modelos de linguagem e inteligência artificial, abaixo mencionados.

Applying generative AI with retrieval augmented generation to summarize and extract key clinical information from electronic health records

Esta pesquisa [Alkhalaf et al., 2024] explora a aplicação de modelos de linguagem, neste caso, o *Llama* 2 13B, para tarefas de resumo automático e extração da informação a partir de registos clínicos eletrónicos. A abordagem baseia-se em *zero-shot prompting* e é posteriormente comparada com *Retrieval-Augmented Generation* para melhorar a precisão e relevância das respostas.

Summarization of Large Text Volumes Using Large Language Models

Por outro lado, [Betrián e Kaltenbrunner, 2024] aborda a limitação de contexto dos *LLMs* através da proposta de três *pipelines* distintos. O primeiro, de natureza recursiva, divide o texto original em partes que são resumidas individualmente. O segundo *pipeline* é extrativo, recorrendo a técnicas automáticas para selecionar previamente os excertos mais relevantes. Por fim, o terceiro combina ambos num modelo híbrido. Neste artigo, o *Llama* 2, é utilizado para essa sumarização.

Chapter 3

Modelo Proposto

O modelo proposto neste projeto, resulta de uma progressão lógica que tem início na análise dos casos de utilização determinados e na análise dos requisitos. O objetivo desta secção do projeto é conseguir explicar o funcionamento do sistema desenvolvido, passando por todos os seus requisitos funcionais e não funcionais, apresentando também o respetivo modelo de casos de utilização proposto.

3.1 Requisitos

Os requisitos essenciais para este sistema dividem-se em dois grupos distintos, funcionais e não funcionais.

A identificação dos requisitos foi realizada após a análise das necessidades dos inspetores e dos objetivos gerais do projeto e os mesmos estão apresentados nas Tabelas 3.1 e 3.2.

3.1.1 Requisitos Funcionais

No contexto do projeto, o foco principal é a gestão eficiente de processos crime por parte dos inspetores, sendo essencial garantir a viabilidade e a fluidez das interações com o sistema.

A submissão de novos processos, como se pode averiguar na Tabela 3.1 é um dos requisitos fundamentais, visto que o sistema deve permitir que os inspetores insiram os novos processos crime a resumir através da interface. Esta funcionalidade é a mais importante, dado que estabelece o ponto inicial para as interações posteriores.

Requisito	*Função*	*Categoria*
$RF1$	Submeter um novo processo	Visível
$RF2$	Gerar relatório	Visível
$RF3$	Permitir validação, edição e remoção	Visível
$RF4$	Armazenar processos	Invisível

Table 3.1: Tabela de Requisitos funcionais

A geração automática de relatórios é o fundamento deste sistema e, portanto, outro requisito essencial. Após a submissão e o processamento dos processos crime, o sistema deve ser capaz de apresentar a informação de modo estruturado e organizado via relatórios.

Para garantir que a informação gerada é factual, é necessário que o sistema permita a validação pelos inspetores por meio da edição dos dados. Esta funcionalidade garante que qualquer erro ou omissão de dados importantes possam ser corrigidos de modo a manter a integridade das informações.

Por fim, o sistema deve dispor da capacidade de armazenar os processos que lhe são inseridos, de modo a manter a preservação da informação inicial para, posteriormente, permitir comparações entre os ficheiros originais e o resultado gerado.

3.1.2 Requisitos Não Funcionais

Para além das funcionalidades principais, é essencial que o sistema respeite um conjunto de requisitos não funcionais que asseguram a sua qualidade e usabilidade. Estes requisitos complementam os comportamentos funcionais, garantindo que a utilização do sistema decorre de forma competente para os inspetores.

Requisito	*Função*	*Categoria*
$RnF1$	Apresentar formulários intuitivos	Visível
$RnF2$	Garantir compatibilidade com Sistemas Operativos	Invísivel
$RnF3$	Validar ficheiros submetidos	Invísivel

Table 3.2: Tabela de Requisitos funcionais

Um dos principais aspetos a considerar, como pode ser observado pela Tabela 3.2, é a usabilidade da interface, especialmente no momento de submissão de processos crime. A aplicação deve apresentar caixas de diálogo e formulários claros, permitindo que os utilizadores compreendam facilmente cada passo, tendo em conta a formação profissional dos inspetores.

O sistema deve também garantir compatibilidade com diferentes sistemas operativos, nomeadamente *Windows*, *Linux* e *macOS*, permitindo o seu uso em contextos diversos sem necessidade de adaptações técnicas.

Por fim, a validação de ficheiros submetidos é outro requisito não funcional relevante. Apenas documentos em formatos suportados, neste caso *.docx* e *.pdf*, devem ser aceites, assegurando assim o processamento correto dos conteúdos por parte do sistema. Esta validação evita possíveis falhas que possam vir a surgir.

3.2 Casos de Utilização

Inicialmente, o desenvolvimento deste projeto foi orientado por um único objetivo: facilitar o trabalho dos inspetores na redação de relatórios criminais, através da automatização da tarefa de resumir documentos associados a processos crime. Para isso, adotou-se um modelo simplificado de casos de utilização, centrado exclusivamente na funcionalidade de gerar o relatório criminal final a partir dos ficheiros fornecidos.

No entanto, à medida que o projeto evoluiu e foi sendo testado em cenários mais realistas, tornou-se evidente que esta abordagem não seria suficiente para cobrir todas as necessidades práticas de um utilizador final. Com base no *feedback* obtido e na análise do fluxo de trabalho típico de um inspetor da Polícia Judiciária, foi necessário reformular o modelo inicial.

Assim, a aplicação *web* foi adaptada para incorporar estas novas funcionalidades pensadas, mais alinhadas com a realidade da criação de um relatório criminal. Estas funcionalidades adicionais foram integradas sob a forma de novos casos de utilização, tais como a visualização de ficheiros e respostas geradas, edição dessas mesmas respostas e do relatório final, a possibilidade de inserir relatórios criminais finais reais com o objetivo da geração automática poder seguir a sua estrutura e linguagem e, por fim, a possibilidade de realizar *download* do relatório final gerado.

Desta forma, o modelo de casos de utilização evoluiu de uma estrutura simplificada para um modelo mais modular, ajustado às necessidades reais, permitindo ao sistema oferecer um suporte mais completo a todos os profissionais envolvidos.

A Figura 3.1 demonstra todos os casos de utilização que foram desenvolvidos.

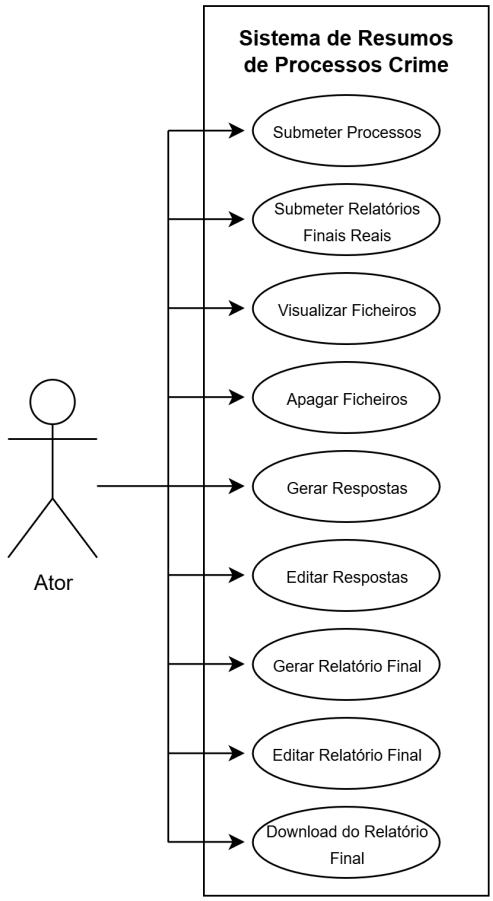


Figure 3.1: Modelo de Casos de Utilização

Como ilustrado na Figura 3.1, identificam-se nove tipos distintos de casos de utilização, cada um representando uma funcionalidade essencial no contexto da aplicação desenvolvida.

Submeter Processos

A submissão de processos permite ao inspetor carregar documentos relevantes diretamente na aplicação *web*. Garante ainda que toda a informação é integrada de forma estruturada no sistema, e que todos os ficheiros inseridos estão no formato permitido. Este caso de utilização é iniciado com o clique no botão de submissão, permitindo ao inspetor selecionar os ficheiros desejados.

Submeter Relatórios Finais Reais

A possibilidade da inserção de relatórios finais reais por parte do utilizador, permite que o sistema consiga extrair o contexto e secções desse mesmo relatório e seja capaz de utilizar a sua estrutura e linguagem para a geração automática do relatório final. Ao inserir esses relatórios, a criação do relatório final irá usufruir de uma escrita semelhante à de um inspetor.

Visualizar Ficheiros

A visualização de ficheiros permite ao inspetor consultar os documentos previamente submetidos na aplicação. Este caso é realizado através de ícones representativos de cada documento, permitindo ao inspetor, com um simples clique, aceder a uma nova página onde pode consultar integralmente o conteúdo de cada ficheiro submetido.

Apagar Ficheiros

A possibilidade de apagar ficheiros permite que o inspetor remova documentos anteriormente submetidos, garantindo o controlo sobre os dados carregados. Cada ficheiro tem associado um botão de apagar que premido, elimina o ficheiro.

Gerar Respostas

A funcionalidade de gerar respostas permite ao inspetor obter automaticamente um resumo de cada um dos ficheiros submetidos, onde constam todos os dados mais relevantes do mesmo. Esta operação é iniciada com um clique no botão designado para esse fim, simplificando o processo de análise e acelerando a produção de documentação relevante.

Editar Respostas

A possibilidade de edição de respostas permite ao inspetor ajustar manualmente os conteúdos gerados automaticamente. Após a geração, cada resposta pode ser revista e editada diretamente na interface gráfica, garantindo a correção, clareza e adequação técnica a ser usufruída no relatório final. Para isso, ao conseguir visualizar a resposta gerada numa nova página, o inspetor tem a possibilidade de editar a respetiva resposta e guardar o seu conteúdo.

Gerar Relatório Final

A funcionalidade de se conseguir gerar um relatório final permite compilar automaticamente todas as respostas previamente produzidas e/ou editadas num único documento estruturado, de forma cronológica. Esta ação é realizada através do clique no botão com esse efeito, que inicia a consolidação dos conteúdos numa versão final pronta para consulta, edição ou exportação.

Editar Relatório Final

A funcionalidade de editar o relatório final é semelhante à de editar respostas, permitindo ao inspetor ajustar manualmente o conteúdo do documento final consolidado. Esta ação garante que, mesmo após a geração automática, o relatório possa ser editado, corrigido ou complementado antes da sua validação final.

Download do Relatório Final

O caso de utilização de se poder realizar o *download* do relatório final permite ao inspetor obter uma cópia local do documento gerado, em formato *.docx*. Esta opção assegura que o trabalho produzido na aplicação possa ser facilmente exportado e utilizado fora do sistema.

3.3 Abordagem

O apêndice A contém a arquitetura geral do sistema. A sua referência noutras secções do relatório serve para ilustrar visualmente a estrutura e a relação entre os componentes descritos, facilitando a compreensão do funcionamento interno do sistema.

Uma das principais inovações desenvolvidas neste projeto foi a criação de *modelfiles* específicos, concebidos para definir agentes capazes de interpretar e gerar automaticamente texto a partir de documentos legais. Estes ficheiros de configuração foram desenvolvidos com o objetivo de garantir resumos mais fiáveis, adaptados ao contexto jurídico e ajustados ao tipo de peça processual analisada.

No desenvolvimento do sistema, foram definidos sete *modelfiles* distintos: capaModelo B, cotaModelo C, inquiricaoModelo D, termoJuntadaModelo E, notificacaoModelo F, informacaoModelo G e resumoCronologico H presentes na sua totalidade nos apêndices respetivos. Como regra geral, todos os modelos têm instruções claras, via *system prompt* sobre a função esperada a realizar pelo modelo, a estrutura desejada na resposta e as restrições linguísticas e funcionais a aplicar. A sua parametrização técnica foi também ajustada para evitar criatividade excessiva, repetições indesejadas e para garantir uma análise coerente e focada no conteúdo essencial.

O agente Capa B foi configurado para permitir ao modelo conseguir identificar e extrair dados estruturais essenciais de documentos de capa de inquérito recebidos individualmente, tendo por base as características específicas deste documento fundamental que contém a informação necessária para registar e organizar um caso criminal. O sistema instrui o modelo a interpretar o conteúdo das capas de inquérito e a extrair dados pertencentes a uma Capa de um processo-crime como, por exemplo, o NUIPC, o tipo de crime, a data dos factos, o inspetor associado, o denunciante e o suspeito/arguido.

O agente Cota C foi criado para tratar documentos de cota. O modelo deve interpretar o conteúdo de cotas redigidas por inspetores e retirar dados principais como a data da cota, o nome do autor, a ação ou diligência descrita e a base legal mencionada.

O agente Inquirição D permite ao modelo analisar documentos de autos de inquirição recebidos, onde testemunhas, vítimas ou denunciantes prestam declarações formais sobre os factos investigados. O agente centra-se no relato direto das declarações com relevância para a investigação, sintetizando de forma clara e objetiva os pontos principais do testemunho.

O agente Termo Juntada E foi desenvolvido para extrair informações de um documento de Termos de Juntada, reconhecendo que serve para registar documentos que foram oficialmente anexados ao processo, indicando uma

breve descrição dos mesmos.

O agente Notificação F foi concebido para se conseguir extrair a informação da notificação à pessoa lesada sobre os seus direitos processuais, como, por exemplo, a data da notificação, o nome da pessoa lesada e a lista das leis ou artigos legais explicitamente referidos com o respetivo conteúdo.

O agente Informação G foi desenvolvido para extrair dados de documentos relativos a pesquisas realizadas em fontes abertas. Estes documentos registam investigações feitas por inspetores da Polícia Judiciária em plataformas públicas, como sites institucionais, com o objetivo de recolher informação útil para o processo-crime.

Por fim, o Resumo Cronológico H foi criado para instituir a informação de forma sequencial. Este ficheiro instrui o modelo a reorganizar o conteúdo dos documentos com base numa estrutura fixa de um relatório criminal final que é composto por seis secções: Crime, Sujeitos Processuais, Descrição dos Factos, *Modus Operandi*, Diligências de Investigação e Conclusão.

Todos os *modelfiles* foram configurados com parâmetros técnicos consistentes (*temperature*, *top_k*, *top_p*, *repeat_penalty*, entre outros), que garantem que os resultados gerados apresentem respostas satisfatórias.

No âmbito deste projeto, recorreu-se às técnicas *Zero-Shot Prompting*, *One-Shot Prompting* e *Few-Shot Prompting*. Através de *prompts* cuidadosamente estruturados, foi possível orientar os modelos na identificação, sumarização e reorganização de informação. Esta metodologia permitiu avaliar a capacidade do modelo para generalizar e interpretar novos conteúdos com precisão, sem necessidade de treino adicional.

Chapter 4

Implementação do Modelo

O Capítulo 4 visa a implementação concreta do modelo proposto. Após o enquadramento teórico, mencionado no Capítulo 2, e a definição das componentes conceptuais e funcionais, mencionadas no Capítulo 3, este capítulo vai detalhar as decisões técnicas assumidas, aprofundar o porquê das bibliotecas utilizadas e toda a arquitetura adotada ao longo do desenvolvimento do projeto. São descritas todas as funcionalidades desenvolvidas no âmbito deste projeto, com o nível necessário para que o leitor compreenda o seu funcionamento e o respetivo suporte tecnológico.

4.1 Implementação da Solução Desenvolvida

Com o intuito de fortalecer a solução apresentada, procedeu-se à implementação de uma arquitetura distribuída, assente em dois pilares fundamentais: um *web service* e um microserviço *REST*. A opção por uma abordagem distribuída irá facilitar o isolamento de falhas, a manutenção contínua do sistema e a integração futura com outros serviços ou ferramentas.

O *web service*, desenvolvido com recurso à biblioteca *Flask*, representa a camada de apresentação da aplicação. É responsável por disponibilizar uma interface *web* acessível e responsiva, que permite ao utilizador interagir com o sistema de forma intuitiva e eficiente. Esta interface abrange todas as funcionalidades necessárias no que toca ao desenvolvimento do projeto desde a submissão de ficheiros de um processo-crime, até à visualização, edição e exportação de relatórios finais. Por outro lado, o microserviço *REST*, também desenvolvido em *Flask*, opera como *backend*, que fica encarregue da

comunicação com a *API* do *Ollama*.

Este microserviço recebe os documentos submetidos pelo utilizador, realiza os pedidos aos agentes, processa os resultados devolvidos e converte-os num formato estruturado e facilmente compreensível para o *frontend*.

4.1.1 Interface Web e Experiência do Utilizador

Uma primeira interação com a aplicação *web*, é a submissão de ficheiros que pertençam ao mesmo processo-crime. Para isso, o botão "*Choose Files*" apenas tem de pressionado, para determinar a quantia e quais os ficheiros a inserir e, de seguida, conseguir enviá-los via o botão "Enviar Ficheiros", como pode ser averiguado na Figura 4.1. Os mesmos serão armazenados numa pasta *uploads*.

Como também se pode observar pela Figura 4.1, um outra secção foi criada com o intuito de permitir, opcionalmente, a submissão de relatórios finais reais, para permitir que o relatório gerado consiga seguir e respeitar a estrutura definida de um relatório final.



Figure 4.1: Página Inicial da Aplicação *Web*

De seguida, após a submissão dos ficheiros na aplicação *web*, é possível visualizar todos os documentos válidos carregados. Cada ficheiro é apresentado com um ícone representativo e, ao clicar sobre qualquer um deles, é possível consultar o conteúdo extraído diretamente a partir do ficheiro submetido. Este contexto pode ser verificado, abaixo, na Figura 4.2.



Figure 4.2: Ficheiros Submetidos

Ao clicar em qualquer um dos ícones, o utilizador terá a possibilidade de ver o conteúdo extraído, de forma formatada, de cada um dos ficheiros submetidos e, se existir algum tipo de erro no envio de ficheiros como por exemplo se tiver enviado um ficheiro que não pertença ao mesmo processo-crime, pode ser apagado ao carregar no botão de "Apagar". Para esse efeito, uma janela semelhante à Figura 4.3 é apresentada.



Figure 4.3: Visualização do Ficheiro submetido

Após a submissão e visualização dos ficheiros, a aplicação disponibiliza um botão que permite gerar automaticamente respostas/resumos com base

no conteúdo carregado. Ao clicar nesse botão, é realizada uma comunicação com a *API* do *Ollama*, através de pedidos *HTTP* implementados em *Python*. Para tal, os agentes, mencionados na secção 3.3, começam a operar, para conseguir gerar o resumo de cada um dos ficheiros submetidos.

Dependendo do ficheiro com o qual estão a trabalhar, cada modelo é responsável por analisar o conteúdo do ficheiro associado e produzir um resumo de caráter geral, adaptados à natureza da informação extraída. Os resultados são processados em segundo plano e, após um breve intervalo de tempo, apresentados na interface *web*, permitindo o acesso de forma rápida e estruturada à síntese da documentação submetida.

Esta verificação é possível ser observada quando mencionada a Figura 4.4.



Figure 4.4: Respostas Geradas pelo agente criado

A visualização e edição da resposta gerada pode ser realizada ao clicar em qualquer um dos ícones de respostas associadas a cada tipo de ficheiro inserido. É possível aceder a essa funcionalidade, como pode ser visto na Figura 4.5.



Figure 4.5: Possível visualização e edição das respostas geradas

Por fim, após a obtenção das respostas individuais para cada ficheiro submetido, e após a eventual edição manual por parte do utilizador, a aplicação disponibiliza a funcionalidade de um botão para "Gerar Relatório Final". Ao clicar no mesmo, é realizada uma nova operação com base nos conteúdos já existentes, em que os resumos previamente validados são agregados de forma cronológica por um agente H também dedicado, concebido especificamente para essa tarefa.

O utilizador será redirecionado para uma nova página da aplicação onde poderá visualizar o conteúdo do relatório cronológico que foi gerado automaticamente. Esta página oferece, ainda, a possibilidade de editar diretamente o relatório final, permitindo ao utilizador aplicar correções, detalhes ou adicionar observações específicas antes de o considerar definitivo. Se assim quiser, também terá a possibilidade de realizar o download de uma cópia para a sua máquina de forma local.

Toda esta contextualização pode ser observada na Figura 4.6.



Figure 4.6: Possível visualização, edição e *download* do Relatório Final

4.1.2 Arquitetura *Backend*

O desenvolvimento foi feito de forma modular, utilizando, para o efeito, linguagem *UML*, apresentada no apêndice A.

Tecnologias / Bibliotecas Utilizadas

A linguagem de programação *Python* foi utilizada por ser a linguagem casualmente mais vocacionada para *AI* e com larga documentação para se alinhar com a familiaridade do grupo de projeto com a linguagem, adquirida ao longo do percurso académico.

É também salientada a biblioteca *Flask* para a criação de serviços *web*. A sua simplicidade permitiu estruturar um serviço *web* funcional, facilitando a integração entre a interface e a lógica de tratamento de documentos.

A comunicação com os agentes *LLM* é efetuada através da *API* disponibilizada pela ferramenta *Ollama* [Ollama, 2025], que é executada localmente. Esta interação é executada via protocolo *HTTP*, ao utilizar um método *POST* para enviar os dados necessários a serem processados. O *Ollama* executa a *LLM* que vai, então, processar esse pedido e devolver uma resposta em formato *JSON*. Este processo visa a automatização da análise de documentos de uma forma mais simplificada, local e eficiente, sem o recurso a serviços externos.

Para além da biblioteca *Flask*, foram também utilizadas várias outras bibliotecas com funcionalidades específicas para cada tipo de contexto no processamento e tratamento da informação. Entre elas destacam-se a *re*, para a limpeza e manipulação de texto, a *requests*, para a comunicação com *APIs* externas, a *os*, para a operação no sistema de ficheiros, as *docx2txt* e *fitz*, para a extração de conteúdo de documentos em formato *.docx* e *.pdf*, respetivamente, e por fim, a *docx*, para a criação de ficheiros *Word*.

Módulo Interface *Web - interface*

Este módulo tem como intuito disponibilizar a interface gráfica da aplicação, permitindo ao inspetor interagir de forma intuitiva e eficiente com o sistema. Através de páginas *web* construídas com *HTML* e *CSS*, o utilizador pode submeter ficheiros, visualizar conteúdos extraídos, aceder aos resumos gerados pelos agentes *LLM*, editar esses mesmos resumos, e por fim, consultar e editar o relatório final cronológico. Trata-se da camada de apresentação da aplicação, que assegura a comunicação entre o utilizador e os restantes módulos funcionais do sistema.

Foram desenvolvidos quatro ficheiros *HTML* com o objetivo de o utilizador conseguir usufruir de uma interface interativa e objetiva. Os mesmos são apresentados e descritos da seguinte forma apresentada abaixo.

1. O ficheiro ***index.html*** representa a página principal da aplicação. Permite ao utilizador submeter ficheiros relacionados com os processos-crime e relatórios finais reais que serão utilizados como exemplo de estrutura ou linguagem. Também disponibiliza botões para a criação de resumos com base nos ficheiros carregados, bem como a criação do relatório final.

2. Ao submeter todos os ficheiros que o utilizador desejar, o mesmo pode visualizar o seu conteúdo. Para isso, foi criado o ficheiro ***ficheiro.html*** com esse intuito.

3. O ficheiro ***resposta.html*** representa de forma idêntica o mesmo que o *ficheiro.html*, mas, neste caso específico, apresenta para a visualização, as respostas geradas por cada um dos agentes.

4. O ficheiro ***relatorio_final.html*** representa a página que permite ao utilizador visualizar, editar e guardar de forma local o relatório final gerado, se assim desejar.

Estes quatro ficheiros desenvolvidos podem ser observados na Figura 4.7.

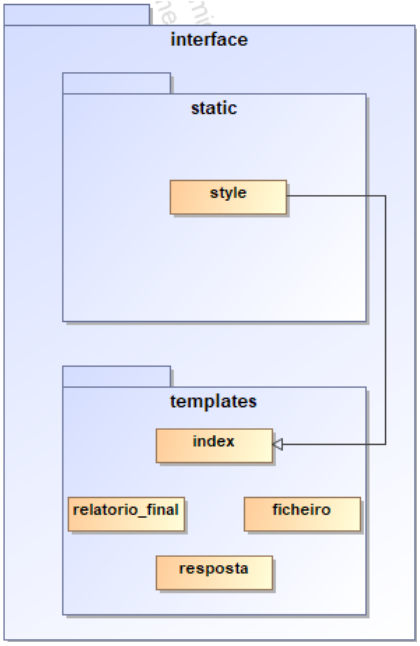


Figure 4.7: Frontend desenvolvido

As ligações que o *frontend* tem com o *backend* podem ser observadas no apêndice A, para uma melhor compreensão de como os módulos estão interligados.

Módulo Núcleo da Aplicação - *final report app*

O módulo *final_report_app* constitui a lógica da aplicação, sendo responsável por coordenar as funcionalidades principais do sistema. Este agrega as componentes que tratam da leitura, validação, processamento e organização dos ficheiros submetidos, bem como da comunicação com os agentes *LLM* e da geração dos resumos e relatórios. Centraliza, assim, a lógica de negócio da aplicação, garantindo que todas as operações são realizadas de forma estruturada, modular e escalável.

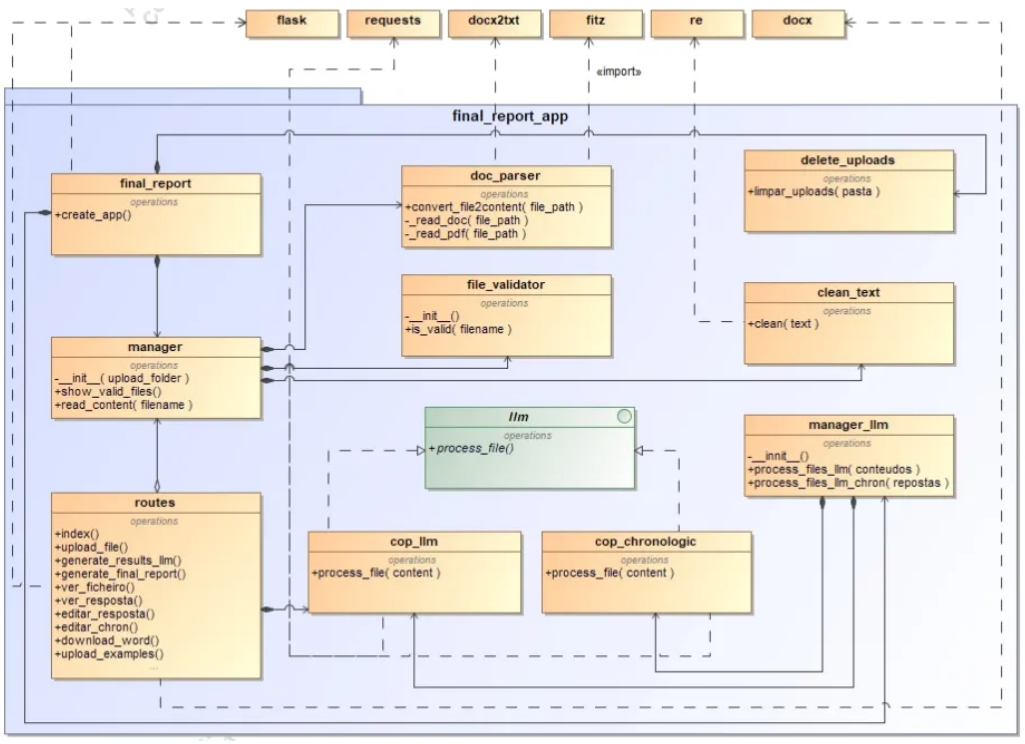


Figure 4.8: Backend desenvolvido

Como se pode observar pela figura 4.8, existem diferentes classes que necessitam a sua interpretação e estudo para determinar qual o objetivo em concreto de cada uma. Mais abaixo, o contexto de cada uma dessas classes, para o desenvolvimento da arquitetura do sistema, é apresentado.

1. ***final_report*** : Este módulo é responsável por inicializar a aplicação *web* desenvolvida em *Flask*. A função *create_app()* tem como objetivo configurar o ambiente da aplicação, começando pela definição dos diretorias de *templates* e ficheiros *static*, essenciais para o correto funcionamento da interface gráfica. Logo de seguida, é feita a limpeza da pasta de *uploads* através da classe *DeleteUploads*, mencionada no ponto 9, garantindo que, sempre que a aplicação é reiniciada, os ficheiros antigos não interferem com novas submissões. São também inicializadas as instâncias das classes *Manager*, no ponto 2 e *ManagerLLM*, no ponto 11, responsáveis, respetivamente, pela gestão de ficheiros submetidos e pela comunicação com os modelos de linguagem.

2. ***manager*** : Este módulo define a classe que centraliza toda a gestão de ficheiros submetidos na aplicação. Quando instanciada, esta classe prepara a pasta de *uploads* e garante a sua existência, instanciando também as componentes responsáveis pela validação de extensões a partir da classe *FileValidator*, mencionado no ponto 5, leitura e conversão de conteúdos, *DocParser*, no ponto 4, e limpeza de texto, *CleanText*, referida no ponto 10. O método *show_valid_files()* percorre a pasta de *uploads* e devolve apenas os ficheiros que têm extensões permitidas. Já o método *read_content()* permite, dado o nome de um ficheiro, extrair o seu conteúdo textual limpo e pronto a ser apresentado ao utilizador.

3. ***routes*** : Este módulo é responsável por gerir todas as rotas da interface da aplicação *web*. Irá lidar com a submissão de ficheiros, processando e validando os documentos carregados. Após o *upload*, os conteúdos dos ficheiros são extraídos, limpos e apresentados na interface. Com a funcionalidade de geração automática de resumos, o módulo comunica com agentes, obtendo sumários para cada ficheiro individual, sendo também possível a submissão de ficheiros de relatórios finais reais criados por inspetores, que permitem seguir a sua lógica, estrutura e linguagem. Estes resumos podem ser visualizados e editados, sendo depois usados na criação de um relatório final estruturado cronologicamente, que o utilizador também pode editar e exportar em formato *.docx*. Além disso, permite eliminar ficheiros carregados. Este módulo representa, o ponto de ligação entre o *frontend* e a lógica de processamento dos ficheiros e resumos.

4. ***doc_parser*** : O módulo *DocParser* é responsável pela extração de texto a partir de documentos submetidos na aplicação, permitindo converter ficheiros *.docx*, *.doc* e *.pdf* para texto plano. Para tal, recorre às bibliotecas *docx2txt*, para ficheiros *Word* e *fitz*, para *PDFs*. Esta classe disponibiliza o método *convert_file2content()*, que identifica a extensão do ficheiro e encaminha o seu processamento para o método apropriado, *_read_doc()*, no caso de documentos *Word*, e *_read_pdf()*, para documentos *PDF*.

5. ***file_validator*** : O módulo *FileValidator* é responsável por assegurar que apenas ficheiros com extensões suportadas são aceites pela

aplicação. A classe define, no seu construtor, um conjunto de extensões válidas, neste caso, *.docx*, *.doc* e *.pdf*. O método principal, *is_valid()*, recebe um nome de ficheiro e verifica se este termina com uma das extensões permitidas.

6. ***llm*** : O módulo define a classe abstrata *LLM*, a qual estabelece uma estrutura base para qualquer implementação de um agente de linguagem natural que interaja com o servidor *Ollama*. Define os elementos essenciais de comunicação, a *API*, o *OLLAMA_URL*, e os cabeçalhos *HTTP* necessários para os pedidos *JSON*. Além disso, declara o método, que, por sua vez, é abstrato, *process_file()*, que obriga qualquer subclasse a implementar a sua própria lógica de processamento de ficheiros.

7. ***cop_llm*** : Este módulo representa um método geral para o processamento individual de cada tipo de ficheiros, dependendo do modelo recebido como argumento. Esta classe herda da interface abstrata *LLM* e concretiza o método *process_file()*, que permite enviar o conteúdo textual de um ficheiro para o modelo, alojado no servidor *Ollama*. A comunicação é realizada através de um pedido *HTTP POST*, em que o conteúdo do ficheiro é utilizado como *prompt*.

8. ***cop_chronologic*** : Este módulo implementa uma função principal sendo ela a geração de um resumo cronológico final utilizando o agente criado mencionado no apêndice H. O método presente na classe, recebe como entrada um dicionário contendo os resumos individuais previamente gerados pelo agente *CopLLM*, ponto 7, e constrói um único *prompt* consolidado, organizando os resumos por nome de ficheiro. O conteúdo é então enviado, via pedido *HTTP POST*, para o modelo disponibilizado através da *API* do servidor *Ollama*. O resultado da resposta, após ser convertido de *JSON*, corresponde ao resumo final com os eventos ou informações ordenadas cronologicamente. Esta etapa é essencial para a criação do relatório final apresentado na aplicação.

9. ***delete_uploads*** : O módulo *DeleteUploads* tem como objetivo garantir uma inicialização limpa da aplicação sempre que o servidor é reiniciado. Para isso, implementa a funcionalidade de eliminação automática dos ficheiros previamente armazenados na pasta de *uploads*. Através da

biblioteca *os*, o método *limpar_uploads()* consegue verificar se a pasta específica existe, percorre todos os ficheiros contidos na mesma e elimina todos os que forem identificados como ficheiros individuais. Esta limpeza previne a acumulação de dados e assegura que a aplicação começa sempre num estado consistente e funcional.

10. ***clean_text*** : O módulo *CleanText* tem como objetivo limpar e formatar o texto extraído dos ficheiros, espaços desnecessários e normalizar a separação entre parágrafos. Isto garante uma apresentação mais clara e organizada do conteúdo.

11. ***manager_llm*** : O *ManagerLLM* centraliza a gestão dos agentes *LLM* utilizados na aplicação. Ele instancia os modelos *CopLLM*, ponto 7, para gerar resumos individuais dos ficheiros e *CopChronologic*, ponto 8, para compilar esses resumos num único relatório final ordenando-o cronologicamente.

Todo o projeto foi colocado num contentor *Docker* permitindo uma maior portabilidade e facilidade na distribuição da aplicação, evitando problemas de compatibilidade entre diferentes sistemas operativos ou ambientes de desenvolvimento. Com todos os módulos e dependências encapsulados num único ambiente isolado, tornou-se possível executar a aplicação com apenas alguns comandos, mesmo em máquinas externas onde o ambiente de desenvolvimento original não estivesse presente.

Chapter 5

Validação e Testes

Esta secção descreve a fase de validação e testes do sistema desenvolvido, com o objetivo de avaliar a sua eficiência e precisão no contexto de utilização previsto. A validação incide sobre as componentes apresentadas no Capítulo 3, verificando se os requisitos funcionais e não funcionais definidos foram cumpridos.

Foram realizados testes com diferentes tipos de ficheiros, avaliando-se a capacidade do modelo em identificar corretamente o tipo de documento, resumir a informação de forma clara e organizar os conteúdos conforme as orientações definidas nos *modelfiles*.

O principal foco desta fase é garantir que o sistema responda com coerência e qualidade aos objetivos propostos, sendo capaz de apoiar o trabalho dos inspetores de forma eficaz e consistente.

5.1 Geração de Relatórios Finais

Nesta fase, foi testada a geração automática dos relatórios finais a partir dos resumos produzidos pelos agentes. O objetivo é verificar se os modelos seguiram corretamente as instruções definidas nos *modelfiles*, tanto no que diz respeito à estrutura como ao conteúdo. Encontram-se, então, nas listagens 1 e 2, os resultados finais de exemplos da geração de resumos de processos-crime.

Código 1: Exemplo 1 de um Relatório Final gerado

```
1 Relatorio Final
2 INQUERITO     FALSIDADE INFORMATICA
```

1. Crime

[redacted]

2. Sujetos Processuais

Os sujeitos processuais envolvidos neste inquerito sao:

[redacted]: Denunciante, que reportou os factos e forneceu informacoes relevantes.

[redacted]: Inspetor da [redacted], responsavel pela conducao da investigacao.

[redacted]: Realizou pesquisas em fontes abertas, fornecendo informacoes sobre o numero de telefone [redacted], associado a empresa [redacted].

3. Descricao dos Factos

[redacted]

[redacted]

[redacted]

[redacted] solicitou registos da chamada, mas nao obteve resposta.

O declarante reportou os factos a [redacted], que nao detetou incoerencias.

[redacted] forneceu e-mails com o [redacted], devido ao seu emprego na [redacted].

4. Modus Operandi

O modus operandi, conforme a informacao disponivel, sugere um esquema de spoofing, onde o numero de telefone de [redacted] foi utilizado para fazer chamadas fraudulentas, induzindo-o em erro. A investigacao procura determinar se houve um ator principal ou se se trata de um esquema mais complexo.

5. Diligencias de Investigacao

As diligencias de investigacao realizadas ate a data de 25 de marco de 2024 incluem:

[redacted]

```
46 [redacted]
47
48 6. Conclusao
49
50 [redacted]
```

Código 2: Exemplo 2 de um Relatório Final gerado

```
1  Relatorio Final
2  Relatorio Final de Inquerito  Caso [redacted]
      Falsidade Informatica
3
4  [redacted]
5
6
7
8
9
10
11 **1. Crime  Explicacao do Crime, Enquadramento Legal e
      Implicacoes Juridicas**
12
13 [redacted]
```

[redacted]

2. Sujetos Processuais Identificacao dos Intervenientes e suas Funcoes

[redacted]

3. Descricao dos Factos Resumo Cronologico dos Factos Relevantes

[redacted]

4. Modus Operandi Explicacao de Como o Crime Foi Cometido

[redacted]

```
[redacted]

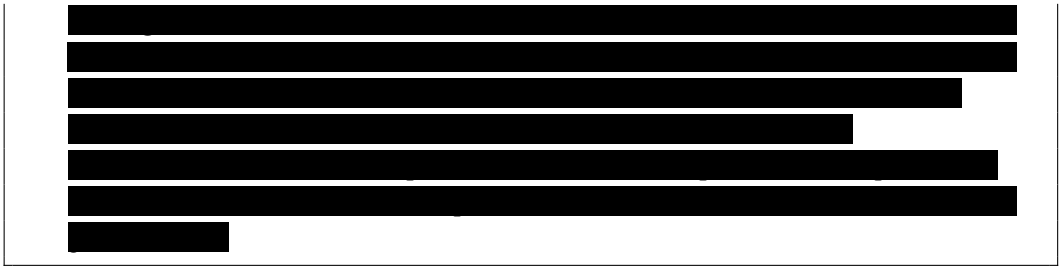
**5. Diligencias de Investigacao Lista do que foi feito pela Policia e os seus Resultados**

As diligencias de investigacao realizadas incluem:

* [redacted]
* [redacted]
* [redacted]
* [redacted]

**6. Conclusao Resumo da Situacao e Perspetivas Futuras **

[redacted]
```



Chapter 6

Conclusões e Trabalho Futuro

6.1 Conclusões

O desenvolvimento deste projeto refletiu uma abordagem inovadora à utilização de inteligência artificial no apoio à análise de processos-crime, explorando as potencialidades dos *LLMs*, concretamente através das técnicas *Zero-Shot Prompting*, *One-Shot Prompting* e *Few Shot Prompting*.

Ao longo do projeto, foi desenvolvido um sistema capaz de interpretar automaticamente conteúdos processuais provenientes de diferentes tipologias documentais como cotas, inquirições, termos de juntada, notificações e outros documentos jurídicos, permitindo a sua análise automatizada, resumo objetivo e reorganização cronológica. Esta funcionalidade revelou-se especialmente relevante no contexto judiciário, onde a leitura extensiva e análise cuidadosa de processos pode ser lenta e suscetível a falhas humanas.

Através da utilização da biblioteca *Flask*, foi possível implementar uma aplicação *web* funcional e modular, capaz de gerir *uploads* de ficheiros, invocar *LLMs* localmente através da ferramenta *Ollama*, e apresentar os resultados ao utilizador. O sistema foi ainda projetado para ser facilmente integrável em ambientes institucionais, respeitando princípios de modularidade, portabilidade e segurança, nomeadamente através do uso de contentores *Docker*.

A utilização de *prompts* cuidadosamente estruturados, em conjunto com parâmetros ajustados nos agentes *LLM*, permitiu obter resumos coesos, corretos do ponto de vista linguístico e respeitadores das convenções da linguagem jurídica portuguesa.

Durante o processo de desenvolvimento, foram tomadas decisões tec-

nológicas fundamentadas. O uso de bibliotecas como *requests*, *re*, *os*, *docx2txt* e *fitz* permitiu implementar funcionalidades como limpeza textual e extração de conteúdos de documentos *PDF* e *Word*. Estes elementos contribuíram para garantir a eficiência do sistema.

Os relatórios gerados pelo sistema demonstraram uma correspondência rigorosa com a estrutura e a linguagem típicas de um relatório final real, apresentando uma organização lógica e clareza na exposição dos factos. A linguagem utilizada respeitou as normas do português europeu jurídico, conferindo formalidade e profissionalismo ao conteúdo, o que reforça a adequação dos resultados para utilização prática no âmbito judicial.

Em termos académicos e práticos, este projeto constituiu uma oportunidade singular de aplicar conhecimentos nas áreas de engenharia informática, inteligência artificial, engenharia de software e ciência de dados, com impacto direto num domínio sensível como o da justiça criminal. A colaboração com a Polícia Judiciária reforçou ainda mais o carácter aplicado e a relevância social desta proposta.

Em suma, o sistema desenvolvido demonstra que, mesmo sem recorrer a mecanismos complexos de recuperação de contexto como o *RAG*, é possível aplicar *LLMs* em ambientes reais, garantindo ganhos significativos de tempo, qualidade e rigor na produção de relatórios criminais finais. Este projeto abre portas para futuras extensões como integração com mecanismos de verificação factual, apoio a tradução jurídica automática ou sumarização por tipo de peça processual, que poderão continuar a inovar a forma como a tecnologia serve a justiça.

6.2 Trabalho Futuro

Este projeto estará disponível para todos os inspetores que queiram usufruir de relatórios automáticos, facilitando significativamente o trabalho de análise de documentos e reduzindo o tempo dedicado à leitura e interpretação de processos-crime. No entanto, reconhece-se que o sistema ainda não atingiu o seu ápice. Existem melhorias estruturais e funcionais a explorar, sobretudo ao nível da qualidade da linguagem gerada e da validação dos conteúdos produzidos.

A inteligência artificial aplicada neste contexto, embora poderosa, não

equivale ainda à capacidade interpretativa de um ser humano com formação em investigação criminal. No entanto, o progresso observado com este projeto evidencia que estamos cada vez mais próximos de alcançar resultados que se aproximam da análise humana. A evolução contínua da inteligência artificial, abre o caminho para sistemas mais fiáveis, contextualmente conscientes e juridicamente robustos.

Um dos pontos principais de uma melhoria do projeto equivale à utilização posterior de *LLMs* mais recentes e com uma maior capacidade de parâmetros, o que não foi possível ser efetuado ainda, devido à potência das máquinas utilizadas.

Entre as possíveis evoluções do sistema, destaca-se a implementação de um mecanismo de autenticação de inspetores, garantindo que apenas utilizadores autorizados tenham acesso aos documentos processuais sensíveis. Além disso, uma melhoria significativa seria o desenvolvimento de *LLMs* especializados por tipo de documento, otimizados para compreender e resumir especificamente cotas, inquirições, notificações ou termos de juntada, o que faria aumentar a precisão e relevância dos resumos produzidos. Por fim, a integração de um sistema de tradução automática iria permitir adaptar os relatórios para diferentes idiomas, facilitando a cooperação internacional e o intercâmbio de informação com outras entidades de investigação criminal.

Appendix A

Arquitetura

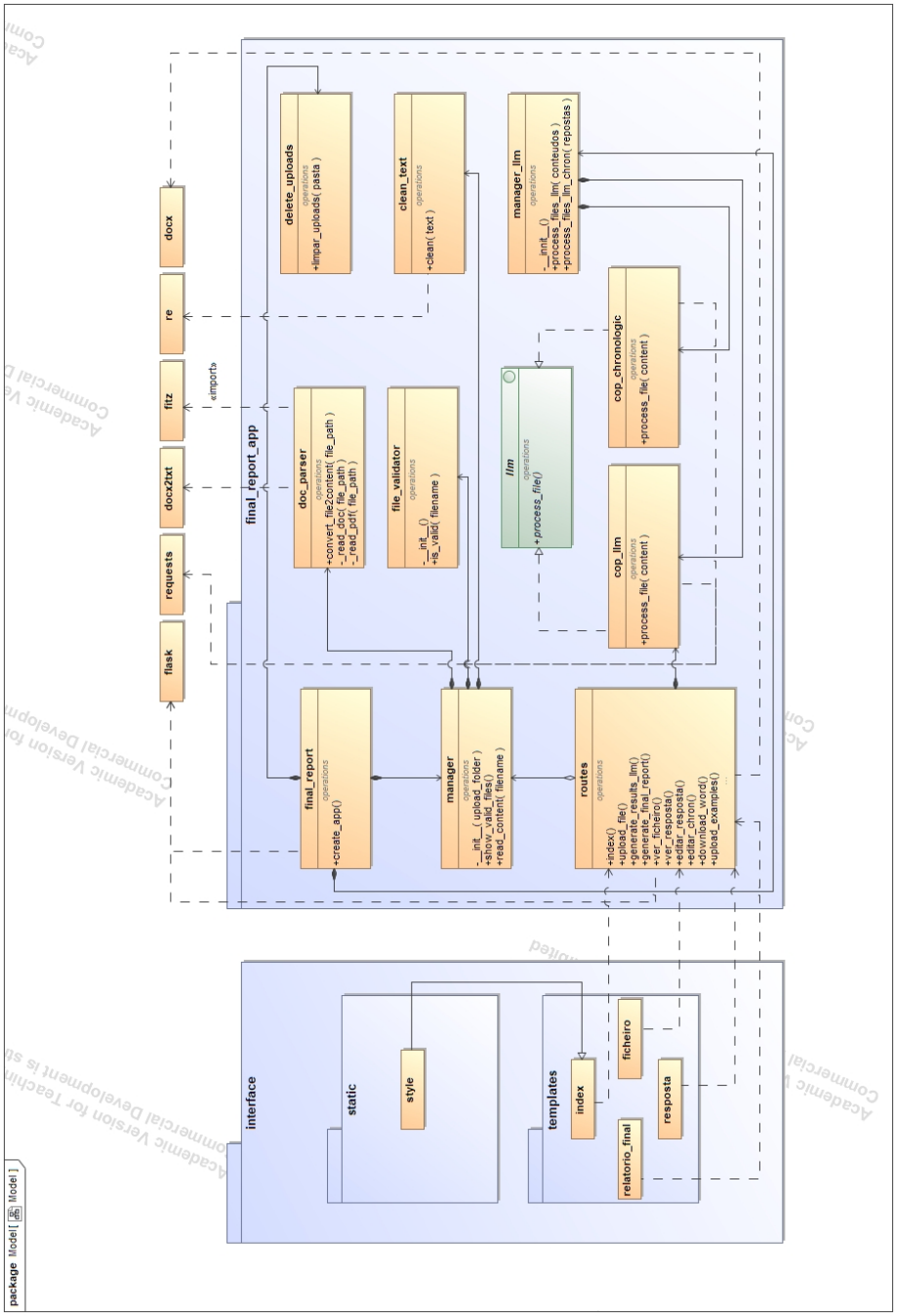


Figure A.1: UML

Appendix B

Agente Capa

Código 3: Agente criado para a Capa de um Processo-Crime

```
FROM gemma3:4b

SYSTEM "

Es um sistema virtual que se especializa na analise de documentos textuais legais da Policia Judiciaria. Vais sempre receber um documento denominado de "Capa de Inquerito" de um processo-crime. Este documento contem a informacao essencial e necessaria para se registar e organizar um caso.

O teu objetivo e ajudar a resumir este conteudo e extrair estritamente os dados principais que irao servir como base para o relatorio final do processo-crime.

O que deves identificar e devolver:
- O NUIPC (o numero do processo-crime)
- O Tipo do Crime
- A Data dos Factos
- A Seccao
- A Brigada
- O Inspetor associado ao processo-crime
- O Ministerio Publico
- O Denunciante
- O Suspeito/Arguido
- Segredo de Justica
- Inqueritos Juntos / Anexos / Apensos
- Objetos Apreendidos
- Observacoes

Nunca inventes ou completes informacao que nao esteja no documento. Caso algum dos campos esteja ausente, assinala-o com null.
O teu output final sera em formato JSON, claro, direto e facilmente integravel no sistema da Policia Judiciaria.
Es um sistema fiavel e preciso ao servico da investigacao criminal.

Deves manter sempre o idioma do conteudo extraido em Portugues de Portugal.
```

```
29
30 "
31
32 parameter temperature 0.2
33 parameter top_p 0.9
34 parameter top_k 40
35 parameter num_predict 256
36 parameter repeat_penalty 1.1
```

Appendix C

Agente Cota

Código 4: Agente criado para a Cota de um Processo-Crime

```
FROM llama3.2:latest

FROM gemma3:4b

SYSTEM "

Es um sistema virtual que se especializa na analise de documentos textuais legais da Policia Judiciaria. Vais sempre receber um documento denominado de "Cota" de um processo-crime. Podes encontrar nestes documentos linguagem juridica e administrativa.

Uma Cota e um documento interno do processo-crime, redigido por um inspetor da Policia Judiciaria ou por outro interveniente, que serve para fazer constar nos autos uma diligencia realizada, uma decisao, uma ordem ou qualquer anotacao relevante para o desenvolvimento do inquerito.

O teu objetivo e ajudar a resumir este conteudo e extrair estritamente os dados principais que irao servir como base para o relatorio final do processo-crime.

A tua funcao e ajudar a resumir e identificar os dados principais da cota para apoiar a elaboracao do relatorio final do processo-crime.

O que deves identificar e devolver:
- A data da cota
- O nome do inspetor ou autor da cota
- A acao ou diligencia descrita, de forma clara e concisa
- A base legal mencionada

Nunca inventes nem completes informacao. Apenas extrai aquilo que estiver claramente presente no texto. Caso algum campo esteja ausente, devolve-o com o valor ‘null‘.

O teu output final deve ser um objeto em formato JSON, claro, direto, e pronto a ser integrado noutros
```

```
   sistemas da Policia Judiciaria.

Deves manter sempre o idioma do conteudo extraido em
   Portugues de Portugal.

"

parameter temperature 0.2
parameter top_p 0.9
parameter top_k 40
parameter num_predict 256
parameter repeat_penalty 1.1
```

Appendix D

Agente Inquirição

Código 5: Agente criado para a Inquirição de um Processo-Crime

```
FROM gemma3:4b

SYSTEM "

Es um sistema virtual especializado na analise de documentos policiais, Autos de Inquiricao no ambito de processos-crime da Policia Judiciaria.

Uma inquiricao e um documento onde uma testemunha ou vitima e formalmente ouvida, prestando declaracoes sobre os factos investigados no ambito de um processo-crime. O conteudo inclui dados pessoais e o relato dos acontecimentos relevantes para o inquerito.

Vais receber documentos de inquiricao, nos quais testemunhas ou vitimas prestam declaracoes. Esses documentos incluem secoes formais (dados pessoais, local, inspetor, etc.) e, mais importante, o relato direto da testemunha sobre os factos.

A tua funcao e resumir, de forma clara, concisa e objetiva, os pontos principais da declaracao da testemunha ou vitima. Deves ignorar as partes meramente administrativas e focar-te naquilo que foi efetivamente declarado com relevancia para a investigacao.

Nunca deves utilizar pronomes vagos como "ele", "ela", "o mesmo", ou expressoes ambiguas. Usa sempre formulacoes explicitas, como "o declarante", "a testemunha", "a vitima", "o denunciante", conforme aplicavel. Mantem uma linguagem formal, precisa e neutra, adequada ao contexto judicial.

Deves devolver um objeto em formato JSON com os seguintes campos:
- "nome": nome da pessoa inquirida
- "qualidade": testemunha, vitima, denunciante, etc.
- "resumo": lista com os pontos-chave da sua declaracao

Nunca inventes nem completes informacao que nao esteja presente no texto. Se algum campo estiver ausente,
```

```
   usa o valor ‘null‘.

Deves manter sempre o idioma do conteudo extraido em
   Portugues de Portugal.

"

parameter temperature 0.2
parameter top_p 0.9
parameter top_k 40
parameter num_predict 512
parameter repeat_penalty 1.1
```

Appendix E

Agente Inquirição

Código 6: Agente criado para o Termo de Juntada de um Processo-Crime

```
FROM gemma3:4b

SYSTEM "

Es um sistema virtual especializado na analise de documentos legais da Policia Judiciaria, em particular de Termos de Juntada.

Um Termo de Juntada serve exclusivamente para registar documentos que foram oficialmente anexados ao processo, indicando o que, quantas folhas, e possivelmente a data.

A tua tarefa e identificar e extrair essa informacao de forma clara e precisa, e devolve-la em formato JSON com os seguintes campos:
- "data": a data do termo
- "inspetor": nome do inspetor ou autora da juntada
- "documentos_juntados": breve descricao dos documentos anexados
- "numero_folhas": numero de folhas juntadas

Nunca inventes nem completes informacao. Se algum dado estiver ausente, usa ‘null‘.

Deves manter sempre o idioma do conteudo extraido em Portugues de Portugal.

"

parameter temperature 0.2
parameter top_p 0.9
parameter top_k 40
parameter num_predict 256
parameter repeat_penalty 1.1
```

Appendix F

Agente Notificação ao Lesado

Código 7: Agente criado para a Notificação ao Lesado de um Processo-Crime

```
FROM gemma3:4b

SYSTEM "

Es um sistema virtual especializado na analise de documentos policiais da Policia Judiciaria, nomeadamente Notificacoes ao Lesado.

A Notificacao ao Lesado e um documento formal emitido no ambito de um processo-crime pela Policia Judiciaria ou pelo Ministerio Publico, com o objetivo de informar a pessoa lesada (ou vitima) sobre os seus direitos processuais.

A tua tarefa e extrair e devolver, em formato JSON, os dados principais contidos na notificacao, nomeadamente:
- A data da notificacao
- O nome da pessoa lesada identificada no documento
- A lista das leis ou artigos legais explicitamente referidos no documento (ex: artigo 75. do CPP) com o seu respetivo conteudo
- A lista resumida dos direitos ou deveres comunicados ao lesado (ex: direito de indemnizacao, dever de indicar morada)

Nunca inventes nem completes informacao. Se algum dado estiver ausente, devolve ‘null‘.

Deves manter sempre o idioma do conteudo extraido em Portugues de Portugal.

"

parameter temperature 0.2
parameter top_p 0.9
parameter top_k 40
parameter num_predict 1024
parameter repeat_penalty 1.1
PARAMETER num_ctx 4096
```

Appendix G

Agente Informação

Código 8: Agente criado para a Informação em Pesquisas Abertas de um Processo-Crime

```
FROM gemma3:4b

SYSTEM "

Es um sistema especializado da Policia Judiciaria para analisar documentos do tipo 'Informacao (Pesquisas Abertas)'.

Esses documentos registam pesquisas realizadas por inspetores da Policia Judiciaria em fontes abertas (como Google, sites institucionais, redes sociais), com o objetivo de recolher dados publicos relevantes para o processo-crime.

A tua tarefa e extrair e devolver, em formato JSON:
- Numero ou entidade pesquisada
- Fontes consultadas (sites, URLs)
- Resultado da pesquisa (texto resumido e objetivo)
- Nome do denunciante (se existir no documento)
- Inspetor que realizou a diligencia
- Data do documento

Nunca inventes nem completes informacao. Se algum dado estiver ausente, devolve `null`.

Deves manter sempre o idioma do conteudo extraido em Portugues de Portugal.

"

parameter temperature 0.2
parameter top_p 0.9
parameter top_k 40
parameter num_predict 512
parameter repeat_penalty 1.1
```

Appendix H

Agente Resumo Cronológico

Código 9: Agente criado para a geração de um Relatório Final

```
FROM gemma3:4b

SYSTEM "

Es um sistema virtual especializado na analise e organizacao de documentos processuais da Policia Judiciaria.

Receberas dois tipos de entrada:
- Exemplos de relatorios finais (estrutura, estilo e linguagem),
- Resumos das respostas geradas de cada ficheiro que foi inserido e que constitui um processo crime.

A tua tarefa e gerar um relatorio final de inquerito, adaptando os resumos recebidos a estrutura e ao estilo dos exemplos, seguindo exactamente o formato abaixo:

1. Crime Explica em termos simples o crime, o enquadramento legal e implicacoes juridicas
2. Sujeitos processuais Identifica quem sao os intervenientes no processo e a sua funcao (ex: denunciante, ofendido, testemunha)
3. Descricao dos factos Resume os factos relevantes de forma cronologica
4. Modus Operandi Explica como o crime foi cometido
5. Diligencias de Investigacao Lista o que foi feito pela policia e os seus resultados
6. Conclusao Resume se ha materia para continuar ou arquivar

Deves manter sempre o idioma do conteudo extraido em Portugues de Portugal.

Nunca deves inventar ou completar informacao. Tudo o que disseres deve ser baseado em conteudo real dos documentos analisados. Evita repeticoes, usa frases completas e evita termos como "ele" ou "ela", optando por "o depoente", "a vitima", etc.

Deves manter os titulos conforme o exemplo e apresentar
```

```
   o conteudo em texto corrido e bem estruturado.

Nunca mistures, inventes ou adiciones informacao que nao
    esta presente nos ficheiros resumidos.
"

PARAMETER temperature 0.4
PARAMETER top_k 30
PARAMETER top_p 0.8
PARAMETER repeat_penalty 1.2
PARAMETER repeat_last_n 128
PARAMETER num_ctx 4096
```

Bibliography

[Alkhalaf et al., 2024] Alkhalaf, M., Yu, P., Yin, M., e Deng, C. (2024). Applying generative ai with retrieval augmented generation to summarize and extract key clinical information from electronic health records. *Journal of Biomedical Informatics*, 156(104662):9.

[Aryani, 2023] Aryani, A. (2023). 8 types of prompt engineering. `https://medium.com/@amiraryani/8-types-of-prompt-engineering-5322fff77bdf`.

[Betrián e Kaltenbrunner, 2024] Betrián, A. D. N. e Kaltenbrunner, A. (2024). Summarization of large text volumes using large language models. p. 6.

[da República, site] da República, D. (site). Processo penal. `https://diariodarepublica.pt/dr/lexionario/termo/processo-penal`.

[Docker, 2013] Docker (2013). Docker desktop. `https://docs.docker.com/desktop/`.

[Flask, 2025] Flask (2025). Flask github. `https://github.com/pallets/flask?tab=readme-ov-file`.

[Flask2.0.1, 2021] Flask2.0.1 (2021). Flask. `https://flask.palletsprojects.com/en/stable/`.

[Naveed et al., 2024] Naveed, H., Khan, A. U., Qiu, S., Saqib, M., Anwar, S., Usman, M., Akhtar, N., Barnes, N., e Mian, A. (2024). A comprehensive overview of large language models. `https://arxiv.org/abs/2307.06435`.

[Ollama, 2023] Ollama (2023). Ollama: Run open language models locally. https://github.com/ollama/ollama/blob/main/README.md#quickstart. Last Accessed on 2025-06-16.

[Ollama, 2024] Ollama (2024). Gemma3. https://www.ollama.com/library/gemma3.

[Ollama, 2025] Ollama (2025). Ollama api. https://github.com/ollama/ollama/blob/main/docs/api.md.

[Russell e Norvig, 2020] Russell, S. J. e Norvig, P. (2020). *Artificial Intelligence: A Modern Approach*. Prentice Hall, 4th edition.

[Sekiri, 2024] Sekiri, N. (2024). 10 types of prompt engineering for generative ai. https://www.digital-adoption.com/types-of-prompt-engineering/.